

Function Declaration

Defines a reusable block of code.

Syntax Example:

```
function greet($name) {  
    return "Hello " . $name;  
}
```

```
echo greet("Ali");
```

Result: Hello Ali

Notes / Tips:

- Functions can take parameters and return values.

Parameters / Return Values

Functions can accept inputs and return outputs.

Syntax Example:

```
function add($a, $b) {  
    return $a + $b;  
}
```

```
echo add(5, 3);
```

Result: 8

Notes / Tips:

- Use return to send value outside the function.
- Without return, use echo inside the function for direct output.

Default / Optional Parameters

Parameters can have default values.

Syntax Example:

```
function greet($name = "Guest") {  
    return "Hello " . $name;  
}  
  
echo greet();
```

Result: Hello Guest

Notes / Tips:

- Default parameters must come **after non-default ones**.

Named Arguments

Pass values to specific parameters by name.

Syntax Example:

```
function profile($name, $age) {  
    return "$name is $age years old";  
}  
  
echo profile(age: 22, name: "Sara");
```

Result: Sara is 22 years old

Notes / Tips:

- Allows skipping parameters or passing in any order.

Anonymous Functions

Functions without a name, can be assigned to variables.

Syntax Example:

```
$greet = function($name) {  
    return "Hello " . $name;  
};
```

```
echo $greet("Ali");
```

Result: Hello Ali

Notes / Tips:

- Stored in variables, passed as arguments.

Arrow Functions

Shorter syntax for simple anonymous functions.

Syntax Example:

```
$double = fn($x) => $x * 2;
```

```
echo $double(5);
```

Result: 10

Notes / Tips:

- Automatically inherits variables from parent scope.

Callback Functions

Functions passed as arguments to other functions.

Syntax Example:

```
function greet($name, $formatter) {  
    return $formatter($name);  
}
```

```
echo greet("Ali", function($the_name) {  
    return strtoupper($the_name);  
});
```

Result: ALI

Notes / Tips:

- Useful for **custom processing or event handling**.
- Can be named, anonymous, or arrow functions.

Recursion

Function that calls itself.

Syntax Example:

```
function count_down($n) {  
    if ($n < 0) return; // when $n becomes less than a 0 the function stops  
    echo "$n\n"; // 5, 4, 3, 2, 1, 0  
    echo count_down($n - 1); // Keeps calling back the function  
}  
echo count_down(5);
```

Result: 5 4 3 2 1 0

Notes / Tips:

- Must have a **base case** to avoid infinite recursion (infinite loop).
- Common in tree or mathematical problems.

Variadic Function

Accepts **any number of arguments**.

Syntax Example:

```
<?php
```

```
function calc(...$num) {  
    // (...$num) return functions argument as an array [2, 3, 4, 1, 5];  
    return implode(' ', $num);  
    // using implode (join() in js) to transform the array to a string  
}
```

```
echo calc(2, 3, 4, 1, 5);
```

Result: 2 3 4 1 5

Notes / Tips:

- Use ...\$var to collect all arguments into an array.
- Perfect for functions where you don't know how many inputs you'll get.